



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
مدل‌های مولد عمیق

تمرین دوم

نام و نام خانوادگی	سید نوید هاشمی
شماره دانشجویی	810101549
تاریخ ارسال گزارش	1404/9/1

فهرست

3.....	سؤال اول : NORMALIZING FLOW
3.....	بخش اول : بخش تئوری
3.....	زیربخش اول
4.....	زیربخش دوم
5.....	بخش دوم : بخش عملی
5.....	زیربخش اول : پیاده سازی
6.....	زیربخش دوم : آموزش مدل و تولید عکس
10.....	زیربخش سوم : آشکارسازی ناهنجاری
15.....	سؤال دوم : CYCLEGAN
15.....	بخش اول : سوالات تئوری
15.....	زیربخش اول : تابع زیان مدل
18.....	زیربخش دوم : معماری و فرایند آموزش مدل
21.....	زیربخش سوم : محدودیت های مدل
21.....	بخش دوم : پیاده سازی عملی
21.....	زیربخش اول : آماده سازی داده ها
22.....	زیربخش دوم : پیاده سازی معماری مدل
22.....	زیربخش سوم : پیاده سازی تابع هدف و آموزش مدل
29.....	مراجع

NORMALIZING FLOW : سؤال اول

بخش اول : بخش تئوری

زیربخش اول

1

$$Z \sim U(0,1)$$

$$P_x(x) = P_z(f^{-1}(x)) |f^{-1'}(x)|$$

$$X = 2Z + 1 \rightarrow Z = \frac{X - 1}{2}$$

$$|f^{-1'}(x)| = \frac{1}{2}$$

$$P_z(Z) = 1 \quad \& \quad 0 \leq Z \leq 1$$

$$P_x(x) = \frac{1}{2} \quad \& \quad 1 \leq Z \leq 3$$

2

$$Z \sim U(0,2)$$

$$P_x(x) = P_z(f^{-1}(x))|f^{-1}'(x)|$$

$$X = \exp(Z) = e^Z \rightarrow Z = \log X$$

$$|f^{-1}'(x)| = \left| \frac{1}{X} \right|$$

$$P_z(Z) = \frac{1}{2} \quad \& \quad 0 \leq Z \leq 2$$

$$P_x(x) = \frac{1}{2X} \quad \& \quad 1 \leq X \leq e^2$$

زیربخش دوم

1

$$x = (x_1, x_2)$$

$$z = (z_1, z_2)$$

$$\begin{cases} z_1 = x_1 \\ z_2 = x_2 + m(x_1) \end{cases} \rightarrow \begin{cases} x_1 = z_1 \\ x_2 = z_2 - m(z_1) \end{cases}$$

$$J = \begin{bmatrix} \frac{\partial x_1}{\partial z_1} & \frac{\partial x_1}{\partial z_2} \\ \frac{\partial x_2}{\partial z_1} & \frac{\partial x_2}{\partial z_2} \end{bmatrix} \rightarrow J = \begin{bmatrix} 1 & 0 \\ ? & 1 \end{bmatrix} \rightarrow \det(J) = 1$$

2

$$x = (x_1, x_2)$$

$$z = (z_1, z_2)$$

$$\begin{cases} z_1 = x_1 \\ z_2 = s(x_1)x_2 + t(x_1) \end{cases} \rightarrow \begin{cases} x_1 = z_1 \\ x_2 = \frac{z_2 - t(z_1)}{s(z_1)} \end{cases}$$

$$J = \begin{bmatrix} \frac{\partial x_1}{\partial z_1} & \frac{\partial x_1}{\partial z_2} \\ \frac{\partial x_2}{\partial z_1} & \frac{\partial x_2}{\partial z_2} \end{bmatrix} \rightarrow J = \begin{bmatrix} 1 & 0 \\ ? & \frac{1}{s(z_1)} \end{bmatrix} \rightarrow \det(J) = \frac{1}{s(z_1)}$$

بخش دوم : بخش عملی

زیربخش اول : پیاده سازی

جزئیات پیاده سازی MADE و بلوک MAF

برای پیاده سازی مدل Masked Autoregressive Flow، کار را از پایه ای ترین جزء، یعنی اعمال محدودیت Autoregressive Property شروع کردم. در ادامه جزئیات پیاده سازی لایه ها و بلوک اصلی را آورده ام:

(1) پیاده سازی لایه Masked Linear

به جای استفاده از لایه های تمام متصل معمولی ($nn.Linear$)، کلاسی به نام Masked Linear تعریف کردم که از کلاس اصلی $nn.Linear$ در pytorch ارث بری می کند.

مکانیزم ماسک: در این کلاس، علاوه بر وزن ها (weight) و بایاس، یک تانسور به نام mask با ابعاد مشابه ماتریس وزن تعریف شد. نکته مهم در پیاده سازی این بود که ماسک با استفاده از متد $register_buffer$ تعریف شد تا جزء پارامترهای قابل یادگیری نباشد و توسط optimizer آپدیت نشود، اما وضعیت آن همراه مدل ذخیره گردد.

Forward: در هر بار محاسبات، پیش از ضرب ماتریسی ورودی در وزن ها، عملیات ضرب درایه ای (Element-wise) بین ماتریس وزن و ماسک انجام می شود. این کار باعث می شود اتصالاتی که نباید وجود داشته باشند (برای حفظ خاصیت autoregressive)، صفر شوند.

(2) ساختار بلوک MADE

بلوک MADE به عنوان یک شبکه عصبی کوچک طراحی شد که وظیفه دارد پارامترهای توزیع شرطی هر پیکسل را تخمین بزند. طبق پیشنهاد مقاله، از یک شبکه ۳ لایه استفاده شد. لایه ورودی ابعاد تصویر فلت شده $(3 \times 128 \times 128)$ را می گیرد و به لایه پنهان با ۵۱۲ نورون می برد. در نهایت، لایه خروجی ابعادی دو برابر ورودی دارد تا برای هر پیکسل دو مقدار میانگین (μ) و اسکیل (α) تولید کند.

برای رعایت دقیق شرط autoregressive، به هر نورون در لایه های ورودی، مخفی و خروجی یک درجه (Degree) نسبت داده شد؛ ماسک اول طوری تنظیم شد که نورون های مخفی فقط به ورودی هایی با درجه کمتر یا مساوی متصل شوند و ماسک آخر طوری تنظیم شد که خروجی های مربوط به پیکسل i ، فقط به نورون های مخفی ای متصل باشند که اطلاعات پیکسل های 1 تا $i - 1$ را دارند.

3) پیاده‌سازی جریان MAF (تک بلوک)

در کلاس MaskedAutoregressiveFlow، ارتباط بین MADE و فرمول تغییر متغیر برقرار شد؛ ورودی x به شبکه MADE داده می‌شود تا خروجی‌های μ و $\log \alpha$ تولید شوند. برای پایداری عددی (Numerical Stability)، روی خروجی α از تابع $\tanh$ استفاده شد تا مقادیر اسکیل کنترل شده باشند.

همچنین در فاز آموزش (Forward)، با داشتن x ، متغیر پنهان z به صورت زیر محاسبه می‌شود:

$$z = (x - \mu) \cdot \exp(-\alpha)$$

و دترمینان ژاکوبین که برای تابع زیان (NLL) ضروری است، به سادگی از مجموع مقادیر $-\alpha$ (یا همان $-\log(\text{scale})$) به دست می‌آید، چرا که ماتریس ژاکوبین به واسطه خاصیت autoregressive، سه گوش است.

همچنین باید اضافه کنم که برای افزایش قدرت مدل و جلوگیری از وابستگی ثابت پیکسل‌ها، بین بلوک‌های متوالی MADE از جایگشت تصادفی ابعاد ورودی استفاده شد تا مدل بتواند وابستگی‌های پیچیده‌تری را بین پیکسل‌های تصویر یاد بگیرد.

زیربخش دوم : آموزش مدل و تولید عکس

1

بر اساس آزمایش‌های انجام شده، نتایج زمانی به شرح زیر ثبت گردید:

زمان کل آموزش (۱۰۰ اپیاک): --- Training Finished in 1:07:40.532150 ---

تعداد کل تصاویر دیده‌شده در طول آموزش $21,900 = 219 \times 100$ تصویر.

زمان متوسط پردازش هر اپیاک (شامل ۲۱۹ تصویر): حدود ۴۰ ثانیه.

زمان تولید عکس:

(لازم به ذکر است که برای این قسمت من یکبار در یک حلقه ۵ تصویر تولید کردم و یکبار تنها یک تصویر)

زمان تولید ۵ تصویر: تقریباً ۴۰ دقیقه (۲۴۰۰ ثانیه) - زمان تولید ۱ تصویر: تقریباً ۸ دقیقه (۴۸۰ ثانیه).

زمان تولید تنها یک تصویر: ۱۷۱۱ ثانیه (تقریباً ۲۸.۵ دقیقه).

تحلیل مقایسه‌ای:

همانطور که مشاهده می‌شود، تفاوت فاحشی میان سرعت آموزش و سرعت تولید وجود دارد. سرعت پردازش در فاز آموزش حدود ۴۰۰۰ برابر سریع‌تر از فاز تولید است. این اختلاف عظیم، هزینه محاسباتی سنگین تولید سریال (پیکسل به پیکسل) در مدل MAF را نشان می‌دهد. در حالی که در آموزش، تمام ۴۹,۱۵۲ پیکسل به صورت موازی پردازش می‌شوند، در تولید، شبکه باید ۴۹,۱۵۲ بار به صورت متوالی اجرا شود. این مشاهده نشان می‌دهد همانطور که ما در اسلاید های درس دیدیم، مدل MAF در فاز آموزش بسیار بهینه و سریع عمل می‌کند، اما در فاز نمونه‌برداری (Sampling) و تولید داده‌ی جدید، به طرز قابل توجهی کند است.

2

کندی قابل توجه در فرآیند تولید تصویر در مقایسه با آموزش در مدل‌های autoregressive مانند Masked Autoregressive Flow (MAF)، ریشه در ساختار احتمالاتی و نحوه مدل‌سازی داده‌ها دارد. در مدل‌های autoregressive، احتمال توأم داده x به صورت حاصل ضرب احتمالات شرطی تک‌تک ابعاد (که در حوزه تصویر ابعاد همان پیکسل‌ها هستند) تجزیه می‌شود:

$$p(x) = \prod_{i=1}^D p(x_i | x_1, x_2, \dots, x_{i-1})$$

این فرمول‌بندی باعث ایجاد تفاوت بنیادی در نحوه پردازش در دو فاز آموزش و تولید می‌شود؛ در واقع در فاز آموزش از آنجایی که تمام پیکسل‌های تصویر واقعی (x) به عنوان ورودی در دسترس هستند، مدل می‌تواند تمام احتمالات شرطی $p(x_i | x_{<i})$ را برای تمام i ها به صورت همزمان و موازی محاسبه کند. به لطف معماری MADE و استفاده از GPU، این عملیات بسیار سریع انجام می‌شود. اما در فاز generation ما هیچ تصویر اولیه‌ای نداریم. برای تولید پیکسل i ام، مدل نیاز دارد که مقادیر تمام پیکسل‌های قبلی (x_1 تا x_{i-1}) از قبل تولید و مشخص شده باشند. این وابستگی sequential باعث می‌شود که نتوانیم از قابلیت پردازش موازی استفاده کنیم. در واقع مدل مجبور است یک حلقه for به طول D (تعداد کل پیکسل‌ها) را طی کند:

1. ابتدا پیکسل x_1 را تولید کند.

2. سپس با استفاده از x_1 ، پیکسل x_2 را تولید کند.

3. ...

4. و این کار را تا تولید آخرین پیکسل x_D ادامه دهد.

با توجه به اینکه تصاویر مورد استفاده $128 \times 128 \times 3$ هستند، تعداد ابعاد $D = 49,152$ است. بنابراین، برای تولید تنها یک تصویر، شبکه عصبی باید 49,152 بار به صورت متوالی اجرا شود که زمان بسیار زیادی را مصرف می‌کند.

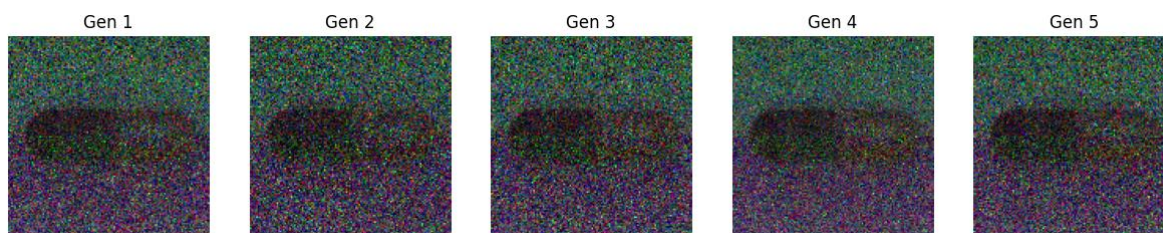


Figure 1 نمونه تصاویر کپسول تولید شده توسط مدل *Masked Autoregressive Flow (MAF)* پس از ۱۰۰ اپیک آموزش.

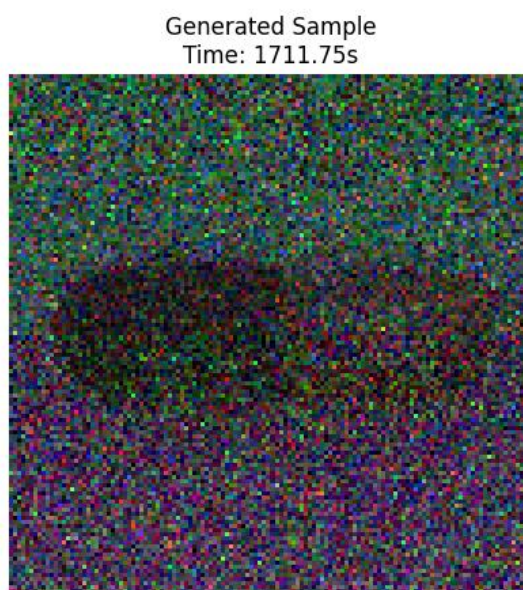


Figure 2 نمونه تصویر تولید شده توسط مدل *MAF* به همراه زمان محاسباتی.

همانطور که مشاهده می‌شود، مدل موفق به یادگیری ساختار کلی، شکل هندسی و موقعیت قرارگیری کپسول شده است. وجود نویزهای دانه دانه در تصاویر ناشی از فرآیند ناپیوستگی زدایی (Dequantization) در حین آموزش است که طی آن، مقادیر گسسته پیکسل‌ها با افزودن نویز تصادفی به یک توزیع پیوسته تبدیل شده‌اند تا با ماهیت مدل‌های Flow سازگار باشند.

همانطور که در آزمایش‌ها مشاهده شد، مدل *Masked Autoregressive Flow (MAF)* در فاز تولید تصویر بسیار کند عمل می‌کند. در مقابل، مدل *Inverse Autoregressive Flow (IAF)* به عنوان یک راهکار جایگزین با ویژگی‌های عملکردی معکوس شناخته می‌شود.

تفاوت بنیادین این دو مدل در جهتی است که وابستگی autoregressive اعمال می‌شود؛ در مدل *MAF* وابستگی روی داده‌های ورودی/خروجی (x) مدل‌سازی می‌شود. یعنی برای محاسبه z_i ،

به $x_{1:i-1}$ نیاز داریم (که سریع است چون x معلوم است)، اما برای تولید x_i ، به $x_{1:i-1}$ نیاز داریم که هنوز تولید نشده‌اند. این باعث می‌شود فرآیند تولید، serial و کند باشد.

اما در مدل IAF وابستگی autoregressive روی متغیرهای پنهان (z) مدل‌سازی می‌شود. فرمول به‌روزرسانی در IAF به صورت زیر است:

$$x_i = z_i \cdot \sigma_i(z_{1:i-1}) + \mu_i(z_{1:i-1})$$

در فاز تولید، بردار نویز z (که از توزیع گوسی نمونه‌برداری شده) کاملاً در دسترس است. بنابراین، مدل می‌تواند تمام پارامترهای μ و σ را برای تمامی پیکسل‌ها به صورت همزمان و موازی محاسبه کند. این ویژگی باعث می‌شود تولید تصویر در IAF چندین مرتبه سریع‌تر از MAF باشد.

و اما اکنون زمان آموزش این دو مدل را مقایسه می‌کنیم:

این برتری در سرعت تولید، به بهای کندی در آموزش به دست می‌آید؛ در آموزش مدل MAF برای محاسبه احتمال $p(x)$ (که هدف آموزش است)، MAF فقط به یک Forward Pass نیاز دارد که به دلیل دسترسی به x ، کاملاً موازی و سریع است؛ اما در آموزش IAF برای محاسبه $p(x)$ ، نیاز به معکوس کردن عملیات آن داریم. از آنجایی که ساختار autoregressive در IAF روی z است، محاسبه z از روی x (که برای آموزش لازم است) تبدیل به یک فرآیند سریال و کند می‌شود.

تصاویر زیر، توضیحات بالا را تصدیق می‌کند.

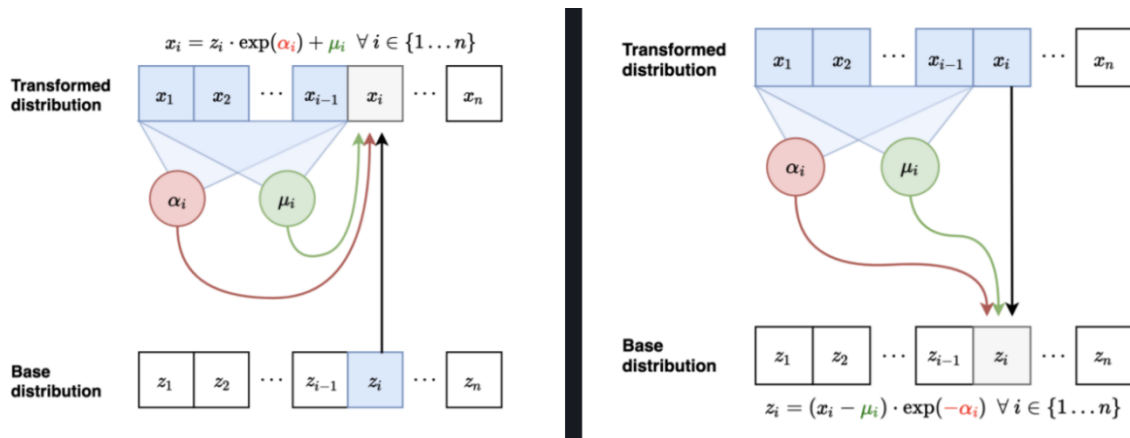


Figure 3 گذر مستقیم (Forward Pass) در مدل MAF (چپ) در مقابل گذر معکوس (Inverse Pass) در مدل MAF (راست)

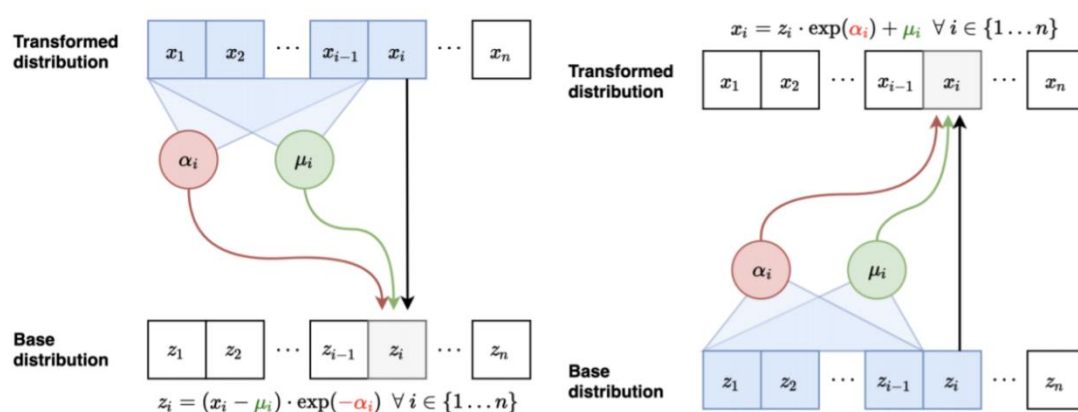


Figure 4 گذر معکوس مدل MAF (چپ) در مقابل گذر مستقیم مدل IAF (راست).

زیربخش سوم : آشکارسازی ناهنجاری

1

استفاده از معیار accuracy برای ارزیابی مدل‌های تشخیص ناهنجاری به دو دلیل عمده گمراه‌کننده و نامناسب است که در ادامه بطور خلاصه به آنها اشاره می‌کنم.

• عدم توازن شدید کلاس‌ها (Class Impalance)

در مسائل تشخیص ناهنجاری، معمولاً تعداد داده‌های سالم (normal) بسیار بیشتر از داده‌های ناهنجار (anomaly) است (مثلاً ۹۹٪ سالم و ۱٪ ناهنجار)، در چنین شرایطی، اگر مدلی همیشه خروجی سالم را پیش‌بینی کند و هیچ ناهنجاری‌ای را تشخیص ندهد، همچنان دقتی برابر با ۹۹٪ خواهد داشت. این دقت بالا کاذب است و مدل عملاً در وظیفه اصلی خود (یافتن آن ۱٪ ناهنجار) شکست خورده است.

• وابستگی به آستانه

خروجی مدل‌های مبتنی بر چگالی (مانند MAF)، یک عدد پیوسته (مانند Likelihood یا anomaly score) است، نه یک برچسب قطعی ۰ یا ۱. برای محاسبه accuracy، مجبوریم یک threshold خاص انتخاب کنیم در نتیجه عملکرد مدل به شدت به انتخاب این آستانه وابسته خواهد بود و accuracy تنها عملکرد در یک نقطه خاص را نشان می‌دهد، نه عملکرد کلی مدل را در تمام سطوح حساسیت.

اکنون با توجه به توضیحات باید معیار دیگری را در نظر بگیریم؛ بهترین و رایج‌ترین معیار برای ارزیابی این مدل‌ها، AUROC (Area Under the Receiver Operating Characteristic Curve) است. این معیار، سطح زیر منحنی ROC (مشخصه عملکرد گیرنده) را محاسبه می‌کند. منحنی ROC، نموداری است که نرخ مثبت واقعی (Sensitivity یا TPR) را در برابر نرخ مثبت کاذب (FPR یا 1 – Specificity) در تمام آستانه‌های تصمیم‌گیری ممکن رسم می‌کند.

مقدار AUROC عددی بین ۰ و ۱ است. این عدد نشان‌دهنده احتمالی است که مدل، به یک نمونه ناهنجار تصادفی، امتیاز ناهنجاری بالاتری نسبت به یک نمونه سالم تصادفی بدهد. مقدار ۰.۵ نشان‌دهنده عملکردی مشابه حدس تصادفی و مقدار ۱ نشان‌دهنده تفکیک کامل و بی‌نقص است.

این معیار برخلاف accuracy، عملکرد مدل را در تمام سطوح حساسیت بررسی می‌کند و نیازی به تعیین دستی آستانه ندارد. همچنین از آنجایی که TPR و FPR به طور مستقل برای کلاس‌های مثبت و منفی محاسبه می‌شوند، نامتوازن بودن تعداد داده‌های سالم و ناهنجار تأثیر منفی بر اعتبار این معیار نمی‌گذارد.

2

anomaly score، یک معیار کمی است که میزان انحراف یک داده‌ی ورودی از توزیع نرمال یاد گرفته شده توسط مدل را نشان می‌دهد. به بیان ساده، این امتیاز به ما می‌گوید که چقدر احتمال دارد که این داده‌ی خاص، متعلق به دسته‌ی داده‌های سالم (normal) باشد.

در مدل‌های مبتنی بر چگالی مانند Normalizing Flows، فرضیه اصلی این است که مدل در طول آموزش، چگالی احتمال $p(x)$ را بر روی داده‌های سالم بیشینه کرده است. بنابراین داده‌های سالم (normal) باید در نواحی با چگالی احتمال بالا قرار بگیرند (High Probability Density)؛ از طرفی داده‌های ناهنجار (anomaly) چون مدل هرگز آن‌ها را ندیده و توزیع آماری متفاوتی دارند، باید در نواحی با چگالی احتمال بسیار پایین قرار بگیرند (Low Probability Density).

از آنجایی که کار با مقادیر احتمال (که اعدادی بسیار کوچک بین ۰ و ۱ هستند) دشوار است و ما می‌خواهیم anomaly ها امتیاز بالاتری داشته باشند، از معیار منفی لگاریتم درست‌نمایی (Negative Log-Likelihood یا NLL) به عنوان anomaly score استفاده می‌کنیم:

$$Anomaly Score(x) = -\log p(x)$$

با توجه به این رابطه، وقتی anomaly score کم است یعنی $p(x)$ زیاد است پس یعنی مدل، این تصویر را بعنوان یک تصویر آشنا و سالم تشخیص داده است؛ در مقابل، وقتی anomaly score زیاد است یعنی $p(x)$ کم است پس یعنی مدل، این تصویر را بعنوان یک تصویر غریبه و ناهنجار تشخیص داده است.

بنابراین، anomaly score یک معیار پیوسته فراهم می‌کند که داده‌های سالم و ناهنجار را در دو ناحیه مجزا از هم قرار می‌دهد. هرچه مدل بهتر آموزش دیده باشد، فاصله بین امتیازات تصاویر سالم و ناهنجار بیشتر خواهد بود. این ویژگی به ما اجازه می‌دهد تا در مرحله ارزیابی (با معیار AUROC) کیفیت این جداسازی را بسنجیم و در مرحله عملیاتی، بسته به حساسیت مورد نیاز، نقطه بهینه‌ای را برای تصمیم‌گیری انتخاب کنیم.

مدل‌های Normalizing Flow (مانند MAF) ، همانطور که در اسلاید های درس هم اشاره شد، بر پایه یادگیری توزیع چگالی احتمال داده‌ها (یعنی $p(x)$) استوار هستند. مکانیسم استفاده از آن‌ها برای anomaly detection به شرح زیر است:

در ابتدا، مدل تنها با استفاده از تصاویر سالم (normal) آموزش داده می‌شود. هدف مدل در این فاز، بیشینه کردن مقدار لگاریتم درست‌نمایی ($\log p(x)$ یا $\log likelihood$) برای این داده‌ها است. به عبارت دیگر، مدل یاد می‌گیرد که تصاویر سالم دارای احتمال وقوع بالا هستند و پارامترهای توزیع را طوری تنظیم می‌کند که چگالی احتمال در فضای تصاویر سالم متمرکز شود.

پس از آموزش، مدل برای داده‌های جدید (که ممکن است سالم یا ناهنجار باشند) به عنوان یک امتیازدهنده عمل می‌کند. برای هر داده ورودی x_{new} ، مدل ، مقدار چگالی احتمال $p(x_{new})$ را محاسبه می‌کند (با استفاده از تبدیل به فضای پنهان z و محاسبه ژاکوبین)؛ اگر x_{new} شبیه به داده‌های آموزشی (normal) باشد، مدل مقدار $p(x_{new})$ بالایی به آن نسبت می‌دهد و اگر x_{new} دارای ویژگی‌های anomaly (مانند ترک، شکستگی یا نویز غیرعادی) باشد که مدل قبلاً ندیده است، این داده در ناحیه‌ای از فضا قرار می‌گیرد که مدل چگالی کمی به آن اختصاص داده است؛ بنابراین مقدار $p(x_{new})$ بسیار پایین خواهد بود.

در نهایت همانطور که در پرسش قبلی اشاره کردم، برای تبدیل این احتمالات به یک معیار تشخیص، از منفی لگاریتم درست‌نمایی (NLL) به عنوان anomaly score استفاده می‌کنیم:

$$Anomaly\ core(x) = -\log p(x)$$

داده‌های سالم ← احتمال بالا ← امتیاز ناهنجاری پایین.

داده‌های ناهنجار ← احتمال پایین ← امتیاز ناهنجاری بالا.

در نهایت، با تعیین یک حد آستانه مناسب روی این امتیاز، می‌توان داده‌های ناهنجار را از داده‌های سالم تفکیک کرد.

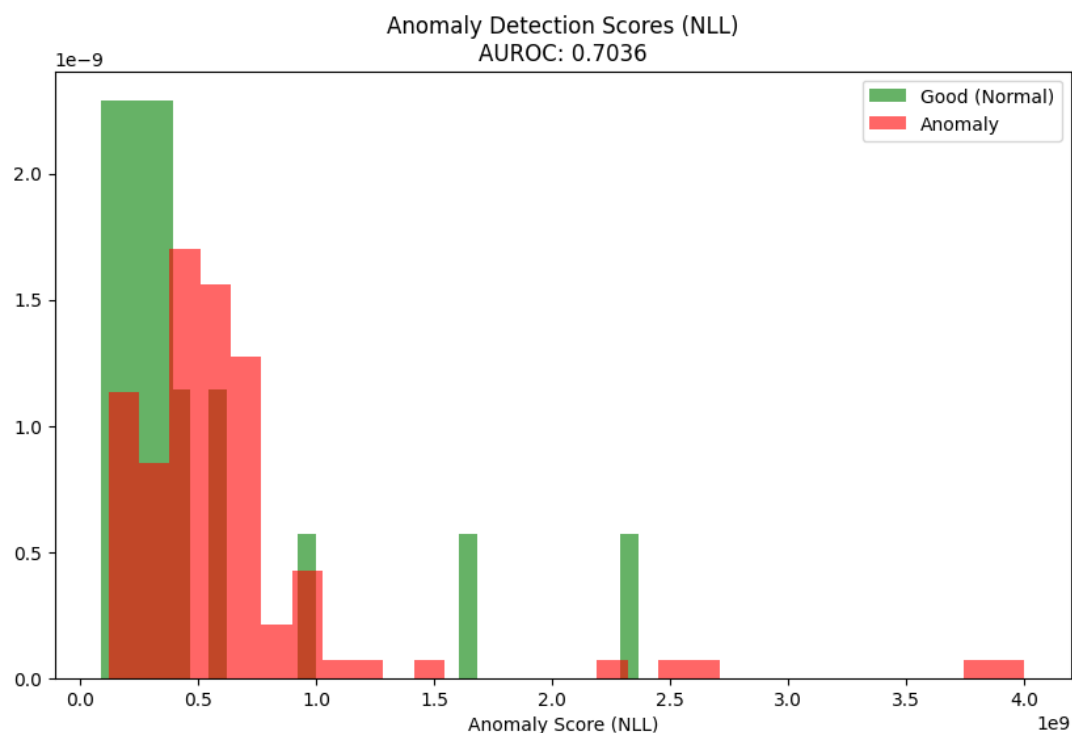


Figure 5 هیستوگرام توزیع امتیازهای ناهنجاری (NLL) برای داده‌های سالم و ناهنجار. رنگ سبز نشان‌دهنده توزیع تصاویر سالم و رنگ قرمز نشان‌دهنده تصاویر ناهنجار است. همانطور که مشاهده می‌شود، توزیع ناهنجاری‌ها به سمت مقادیر بالاتر NLL متمایل است که امکان تفکیک آن‌ها را فراهم می‌کند.

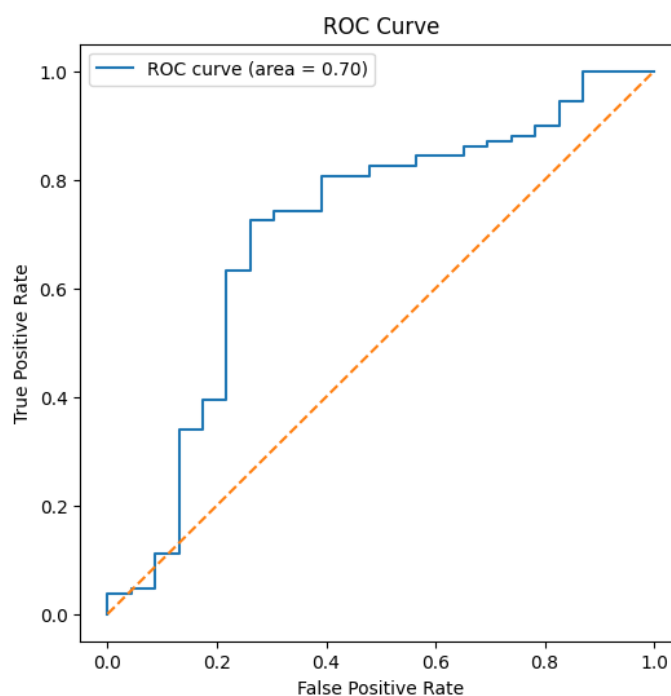


Figure 6 منحنی مشخصه عملکرد گیرنده (ROC) برای مدل MAF. سطح زیر منحنی (AUROC) برابر با ۰.۷۰۳۶ به دست آمده است که نشان‌دهنده عملکرد قابل قبول مدل در تشخیص ناهنجاری‌ها فراتر از حدس تصادفی است.

پاسخ کوتاه این قسمت خیر است؛ استفاده از reconstruction error به روشی که در مدل‌های VAE مرسوم است، برای مدل‌های Normalizing Flow مانند MAF کاربردی و موثر نخواهد بود. دلایل این امر به تفاوت‌های بنیادین در ماهیت این دو مدل برمی‌گردد.

مدل‌های Normalizing Flow بر اساس زنجیره‌ای از توابع Bijective و معکوس‌پذیر طراحی شده‌اند. این ویژگی تضمین می‌کند که نگاشت بین فضای داده (x) و فضای پنهان (z) یک به یک و بدون اتلاف اطلاعات باشد.

در نتیجه، اگر هر تصویری (چه سالم و چه ناهنجار) را به فضای z ببریم و سپس معکوس آن را محاسبه کنیم، دقیقاً به همان تصویر اولیه x بازمی‌گردیم (البته با خطای محاسباتی ناچیز در حد دقت ماشین).

$$x_{reconstructed} = f^{-1}(f(x)) \approx x$$

بنابراین، خطای بازسازی برای همه تصاویر (سالم و ناهنجار) نزدیک به صفر خواهد بود و نمی‌توان از آن به عنوان معیاری برای تمایز استفاده کرد.

اما در مدل‌های VAE، اصطلاحاً به دلیل وجود Information Bottleneck در فضای پنهان و ماهیت تقریبی و lossy بودن فشرده‌سازی، مدل مجبور است تنها ویژگی‌های مهم و پرتکرار (که مربوط به داده‌های نرمال آموزشی هستند) را یاد بگیرد. بنابراین وقتی یک داده ناهنجار به آن داده می‌شود، مدل نمی‌تواند جزئیات ناهنجار را بازسازی کند و نسخه نرمال‌شده‌ای از آن را تحویل می‌دهد که منجر به خطای بازسازی بالا می‌شود. اما در Flow ها، به دلیل حفظ کامل اطلاعات، گلوگاهی برای حذف ویژگی‌های ناهنجار وجود ندارد. پس در Normalizing Flow ها، معیار تشخیص ناهنجاری باید بر اساس چگالی احتمال (Likelihood) باشد، و نه بر اساس خطای بازسازی.

سؤال دوم : CYCLEGAN

بخش اول : سوالات تئوری

زیربخش اول : تابع زیان مدل

1

چالش اصلی در روش‌های نظارتی مانند Pix2Pix، نیاز به داده‌های جفت‌شده (paired training data) است. این یعنی برای آموزش مدل، به مجموعه داده‌ای نیاز داریم که برای هر تصویر ورودی x ، دقیقاً تصویر خروجی هدف y متناظر با آن وجود داشته باشد.

اکنون در پاسخ به اینکه چرا این چالش به راحتی قابل رفع نیست باید بگوییم:

برای بسیاری از تسک‌ها، داده‌های جفت‌شده اصلاً وجود خارجی ندارند. مثلاً برای تبدیل نقاشی‌های مونه به عکس واقعی، هیچ عکس واقعی‌ای وجود ندارد که دقیقاً از همان زاویه و در همان لحظه‌ای که مونه نقاشی کرده، گرفته شده باشد.

حتی در مواردی که تولید داده‌های جفت‌شده ممکن است، این کار بسیار پرهزینه و دشوار است. مثلاً برای تولید جفت‌های روز به شب، نیاز به دوربین‌های ثابت داریم که ماه‌ها در یک مکان نصب شوند تا دقیقاً از یک صحنه در شرایط نوری مختلف عکس بگیرند.

همچنین برای برخی تسک‌ها مثل Object Transfiguration (مثلاً گورخر به اسب که در مقاله نیز به آن اشاره شده)، خروجی مطلوب حتی تعریف مشخصی ندارد (در واقع چگونه می‌توان یک گورخر خاص را دقیقاً به یک اسب خاص تبدیل کرد به طوری که پوزیشن شی دقیقاً حفظ شود؟).

2

ایده اصلی CycleGAN برای رفع این مشکل، قید سازگاری چرخه‌ای (Cycle Consistency Constraint) است. در واقع برای غلبه بر مشکل نبود داده‌های جفت‌شده، ایده اصلی CycleGAN بهره‌گیری از ویژگی Transitivity (بازگشت‌پذیری) در ترجمه است.

فرض کنید ما می‌خواهیم جمله‌ای را از انگلیسی به فرانسوی ترجمه کنیم؛ اگر نتیجه ترجمه را مجدداً از فرانسوی به انگلیسی برگردانیم، باید به همان جمله اولیه برسیم.

در زمینه تصاویر، این مدل فرض می‌کند که یک رابطه زیربنایی بین دو دامنه وجود دارد و نگاشت‌های یاد گرفته شده باید معکوس‌پذیر (bijective) باشند. بنابراین، به جای یادگیری یک طرفه، ما دو نگاشت را همزمان یاد می‌گیریم و این قید را اعمال می‌کنیم که ترجمه رفت و برگشت یک تصویر، باید منجر به بازسازی تصویر اولیه شود.

فرض کنید دو دامنه تصویری X و Y داریم (مثلا X اسب ها و Y گورخر ها)؛ مدل CycleGAN شامل دو تابع نگاشت (generator) است:

$$\begin{aligned} G: X &\rightarrow Y \\ F: Y &\rightarrow X \end{aligned}$$

اکنون قید Cycle Consistency به دو بخش تقسیم می‌شود:

- Forward Cycle Consistency می‌گوید اگر تصویر x از دامنه X را با تابع G ترجمه کنیم و سپس خروجی را با تابع F برگردانیم، باید به x برسیم، رابطه زیر همین مورد را نشان می‌دهد:

$$x \rightarrow G(x) \rightarrow F(G(x)) \approx x$$

- Backward Cycle Consistency را نیز به طور مشابه برای تصویر y از دامنه Y :

$$y \rightarrow F(y) \rightarrow G(F(y)) \approx y$$

اما در فرایند آموزش، برای اعمال این قید، از تابع زیان Cycle Consistency استفاده می‌شود که فاصله بین تصویر اصلی و تصویر بازسازی‌شده را کمینه می‌کند (معمولا با norm L1)؛ عبارت ریاضی آن به فرم زیر خواهد بود:

$$\mathcal{L}_{cyc}(G, F) = \mathbb{E}_{x \sim p_{data}(x)} [\|F(G(x)) - x\|_1] + \mathbb{E}_{y \sim p_{data}(y)} [\|G(F(y)) - y\|_1]$$

3

اگر این قید را حذف کنیم و صرفا بر آموزش تخصصی تکیه کنیم، دو رفتار نامطلوب و اساسی در مدل مشاهده می‌شود:

اولین مورد، مشکل عدم ارتباط معنایی (Random Permutation) است. به عبارتی، تابع زیان تخصصی ($\mathcal{L}_{GAN}$) تنها تضمین می‌کند که توزیع تصاویر تولید شده توسط مولد، با توزیع تصاویر دامنه مقصد (Y) همخوانی داشته باشد. این تابع هیچ تضمینی نمی‌دهد که یک تصویر ورودی خاص مثل x_i به یک تصویر خروجی معنادار و مرتبط مثل y_i نگاشت شود. بدون قید چرخه، شبکه می‌تواند یاد بگیرد که تصاویر ورودی را به هر جایگشت تصادفی از تصاویر دامنه مقصد نگاشت کند (مثلا تصویر یک اسب سفید را به تصویر یک گورخر سیاه که در جهت مخالف ایستاده تبدیل کند)، تا زمانی که خروجی واقعی به نظر برسد.

دومین مورد، مشکل Mode Collapse است. در عمل، بهینه‌سازی زیان تخصصی به تنهایی بسیار دشوار است و اغلب منجر به این پدیده می‌شود. در این حالت، generator، تمام تصاویر ورودی مختلف را نادیده می‌گیرد و آن‌ها را تنها به یک یا چند تصویر خروجی ثابت (که توانسته‌اند discriminator را فریب دهند) نگاشت می‌کند. Cycle Consistency با مجبور کردن مدل به

بازسازی تصویر اولیه، جلوی این اتفاق را می‌گیرد، زیرا اگر تمام ورودی‌ها به یک خروجی تبدیل شوند، بازسازی آن‌ها غیرممکن خواهد بود.

بنابراین با توجه به این موارد، Cycle Consistency تضمین می‌کند که نگاشت‌های یاد گرفته شده (F و G)، نه تنها توزیع‌ها را تطبیق می‌دهند، بلکه معکوس‌پذیر (bijective) بوده و ساختار و محتوای تصویر ورودی را حفظ کنند.

4

تابع زیان نهایی در مدل CycleGAN ترکیبی خطی از زیان‌های تخصصی برای هر دو جهت نگاشت و زیان Cycle Consistency است. رابطه ریاضی کامل این تابع زیان به صورت زیر است:

$$\mathcal{L}(G, F, D_X, D_Y) = \mathcal{L}_{GAN}(G, D_Y, X, Y) + \mathcal{L}_{GAN}(F, D_X, X, Y) + \lambda \mathcal{L}_{cyc}(G, F)$$

در این رابطه، دو ترم اول مربوط به آموزش شبکه برای تولید تصاویر واقعی در دامنه‌های هدف هستند و ترم آخر تضمین می‌کند که تصاویر ترجمه شده قابلیت بازگشت به حالت اولیه را داشته باشند.

در این معادله، ضریب λ به عنوان یک ابرپارامتر کلیدی، نقش کنترل‌کننده اهمیت نسبی بین دو هدف اصلی مدل را ایفا می‌کند؛ یعنی تعادلی میان واقع‌گرایی و تغییر سبک (که توسط زیان GAN force می‌شود) و حفظ محتوا و ساختار ورودی (که توسط زیان Cycle Consistency force می‌شود) برقرار می‌سازد.

با این تفاسیر، چنانچه مقدار λ بیش از حد بزرگ انتخاب شود، مدل اولویت بسیار بالایی برای بازسازی دقیق تصویر ورودی قائل خواهد شد و عملاً تلاش برای فریب دادن discriminator را در اولویت دوم قرار می‌دهد. این عدم تعادل باعث می‌شود که generator به جای اعمال تغییرات لازم برای انتقال سبک، تغییرات بسیار جزئی و محافظه کارانه‌ای روی تصویر اعمال کند تا مطمئن شود تصویر قابل بازگشت است. در نتیجه، خروجی مدل بیش از حد شبیه به تصویر ورودی باقی می‌ماند و عملیات تبدیل دامنه (مثلاً تبدیل نقاشی به عکس) با شکست مواجه می‌شود یا بسیار ضعیف انجام می‌گیرد.

5

زیان همانی یا Identity Loss یکی از مولفه‌های تکمیلی در تابع هدف CycleGAN است که به منظور حفظ ترکیب بندی رنگی و جلوگیری از تغییرات غیر ضروری در تصویر به کار می‌رود. مفهوم این زیان آن است که اگر یک تصویر از دامنه مقصد را به عنوان ورودی به یک generator بدهیم که وظیفه‌اش تولید تصاویر همان دامنه مقصد است، generator باید بتواند آن تصویر را بدون تغییر

باز تولید کند. به بیان ریاضی، برای $G: X \rightarrow Y$ generator، اگر ورودی y که متعلق به دامنه Y است را به آن بدهیم، انتظار داریم خروجی $G(y)$ بسیار شبیه به خود y باشد.

این زیان به طور خاص برای جلوگیری از تغییرات ناخواسته در توناژ رنگی (Tint) و حال و هوای کلی تصویر بسیار موثر است. به عنوان مثال، در تبدیل نقاشی‌های مونه به عکس، بدون استفاده از زیان همانی، generator ممکن است رنگ بندی تصاویر را به دلخواه تغییر دهد؛ مثلاً نقاشی‌هایی را که در روز روشن کشیده شده‌اند به عکس‌هایی با توناژ غروب آفتاب تبدیل کند، صرفاً به این دلیل که چنین تبدیلی از نظر زیان تخصصی و زیان Cycle Consistency قابل قبول است. با افزودن زیان همانی، مدل مجبور می‌شود که ترکیب رنگی ورودی را حفظ کند و تنها سبک را تغییر دهد، نه محتوای رنگی و زمانی تصویر را؛ تصویر زیر از مقاله نیز مربوط به همین بخش است که خروجی مدل را یکبار با استفاده از Identity Loss و یکبار بدون استفاده از آن نشان می‌دهد.

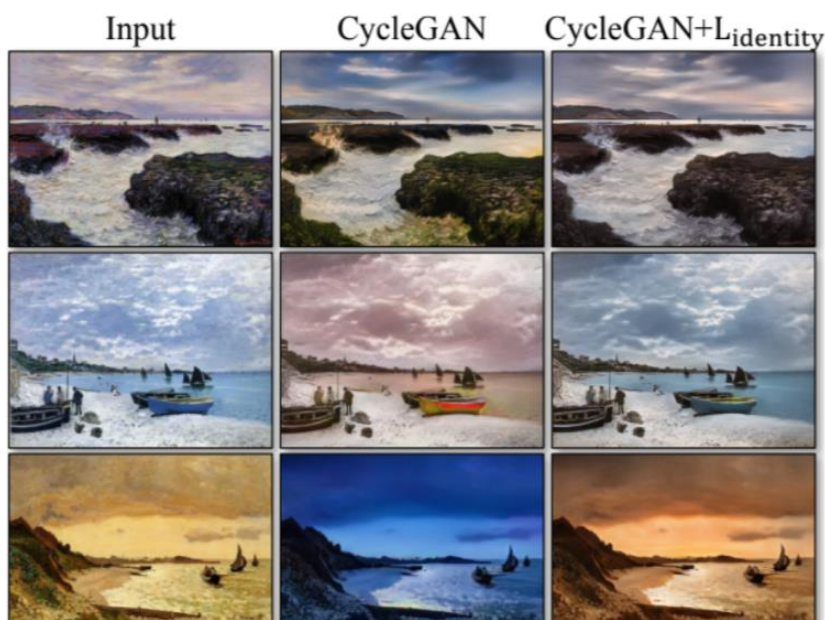


Figure 7 تأثیر *identity mapping loss* روی تبدیل نقاشی‌های مونه ← عکس‌ها. از چپ به راست: نقاشی‌های ورودی، CycleGAN بدون *identity mapping loss*، و CycleGAN با *identity mapping loss*.

زیربخش دوم : معماری و فرایند آموزش مدل

1

از منظر تئوری، مسئله تبدیل تصویر بدون داده‌های جفت‌شده (Unpaired Translation) بطور ذاتی یک مسئله نامقید (Ill-posed) است. اگر سعی کنیم یک شبکه واحد را آموزش دهیم که همزمان هر دو تبدیل را یاد بگیرد (میتوان با یک ورودی اضافی مشخص کرد تبدیل را در کدام جهت انجام دهد)، بهینه‌سازی تابع هدف به شدت پیچیده و ناپایدار خواهد شد. با تفکیک این دو وظیفه به دو شبکه مجزا، ما مسئله را به یادگیری دو نگاشت معکوس‌پذیر (Bijective Mapping) تجزیه

می‌کنیم. این کار به مدل اجازه می‌دهد تا توزیع‌های حاشیه‌ای هر دامنه را به طور مستقل مدل‌سازی کند، در حالی که قید Cycle Consistency تنها به عنوان یک پل ارتباطی بین این دو نگاشت عمل می‌کند تا مطمئن شویم که اطلاعات محتوایی حفظ می‌شود.

از دیدگاه عملی و معماری شبکه نیز، ویژگی‌های تصویری دو دامنه ممکن است کاملاً متفاوت باشند (مثلاً تبدیل عکس‌های هوایی دو بعدی به نقشه‌های انتزاعی). یک شبکه واحد برای یادگیری ویژگی‌های مشترک و غیرمشترک هر دو دامنه به ظرفیت بسیار بالایی نیاز دارد و احتمال تداخل در یادگیری ویژگی‌های اختصاصی هر دامنه افزایش می‌یابد؛ در حالی که استفاده از دو مولد مجزا به هر شبکه اجازه می‌دهد تا به صورت تخصصی بر روی استخراج ویژگی‌ها و تولید تصاویر در دامنه هدف خاص خود تمرکز کند، که این امر همگرایی آموزش را سریع‌تر و پایدارتر می‌سازد.

2

مدل Pix2Pix برای مسائل تبدیل تصویر-به-تصویر با داده‌های paired طراحی شده است، جایی که ورودی و خروجی ساختار مکانی بسیار مشابهی دارند (مانند تبدیل نقشه خطی به عکس ماهواره‌ای). به همین دلیل، مولد در Pix2Pix از معماری U-Net بهره می‌برد. ویژگی بارز U-Net وجود اتصالات پرشی (Skip Connections) است که لایه‌های متناظر در بخش encoder و decoder را مستقیماً به هم وصل می‌کند؛ این اتصالات به شبکه اجازه می‌دهند تا اطلاعات سطح پایین و جزئیات دقیق مکانی را از لایه‌های ورودی مستقیماً به لایه‌های خروجی منتقل کند و از گلوگاه شبکه عبور دهد. این طراحی برای حفظ هم‌ترازی دقیق پیکسلی و جزئیات ساختاری در داده‌های جفت شده حیاتی است.

در مقابل، CycleGAN برای یادگیری با داده‌های unpaired طراحی شده است، جایی که هدف اصلی، اغلب انتقال ویژگی‌های سطح بالاتر مانند سبک یا تغییر ماهیت شیء (مثلاً در تبدیل اسب به گورخر) است و لزوماً تناظر پیکسلی دقیق و مستقیم مانند Pix2Pix وجود ندارد. به همین دلیل، نویسندگان مقاله CycleGAN معماری مبتنی بر ResNet Blocks را که قبلاً توسط جانسون و همکاران برای انتقال سبک عصبی پیشنهاد شده بود، انتخاب کرده‌اند. این معماری شامل یک بخش کاهش ابعاد، تعدادی بلوک ResNet در بخش میانی برای پردازش عمیق ویژگی‌ها (معمولاً ۶ یا ۹ بلوک)، و سپس یک بخش افزایش ابعاد است. عدم وجود اتصالات پرشی مستقیم (مانند آنچه در U-Net است) و استفاده از بلوک‌های ResNet به شبکه اجازه می‌دهد تا تغییرات سراسری‌تر و پیچیده‌تری را روی تصویر اعمال کند و به جای کپی کردن مستقیم جزئیات ورودی، بر روی تغییر محتوا در فضای ویژگی تمرکز نماید. همچنین استفاده از Instance Normalization در این معماری به بهبود کیفیت انتقال سبک کمک شایانی می‌کند.

بنابراین، انتخاب U-Net برای Pix2Pix با هدف حفظ دقیق ساختار در داده‌های جفت‌شده، و انتخاب ResNet برای CycleGAN با هدف انعطاف پذیری بیشتر در تغییرات ظاهری و سبک در داده‌های غیر جفت شده صورت گرفته است.

3

انتخاب معماری PatchGAN برای شبکه discriminator در CycleGAN، با هدف مدل‌سازی دقیق جزئیات فرکانس بالا و بافت تصویر صورت گرفته است. در آموزش مدل‌های تولید تصویر، استفاده از توابع زیان سنتی مانند L1 یا L2 معمولاً منجر به تولید تصاویر تار می‌شود، زیرا این توابع تمایل دارند میانگینی از تمام خروجی‌های محتمل را یاد بگیرند و در بازسازی فرکانس‌های بالا (لبه‌های تیز و جزئیات دقیق) ضعف دارند. در مقابل، معماری PatchGAN فرض می‌کند که وابستگی بین پیکسل‌ها تنها در یک همسایگی محلی معنادار است و پیکسل‌هایی که فاصله زیادی از هم دارند، مستقل از یکدیگرند. بنابراین، به جای اینکه شبکه سعی کند کل تصویر را به عنوان یک واحد real یا fake قضاوت کند، تصویر را به تعدادی Overlapping Patches تقسیم می‌کند و روی هر patch به صورت جداگانه تصمیم می‌گیرد.

خروجی این نوع discriminator یک عدد واحد نیست، بلکه یک ماتریس دو بعدی از اعداد است که هر درایه آن نشان دهنده احتمال واقعی بودن یک ناحیه خاص از تصویر ورودی است. به عنوان مثال، در یک PatchGAN با میدان دید 70×70 ، هر سلول خروجی بیانگر این است که آیا آن patch ۷۰ پیکسلی مربوطه از نظر بافت و ساختار محلی، واقعی به نظر می‌رسد یا خیر. در نهایت، با میانگین‌گیری از تمام این امتیازات محلی، امتیاز نهایی واقع‌گرایی تصویر به دست می‌آید. این رویکرد باعث می‌شود discriminator تمرکز خود را بر روی اصلاح جزئیات ریز و بافت‌های تصویر بگذارد، در حالی که ساختار کلی و فرکانس‌های پایین توسط تابع زیان cycle consistency کنترل می‌شوند. همچنین، این ساختار کاملاً کانولوشنی است، که باعث می‌شود تعداد پارامترها کمتر شده و بتوان آن را روی تصاویری با ابعاد دلخواه اعمال کرد.

4

همانطور که در صورت سوال اشاره شده، در آموزش شبکه discriminator، به جای استفاده انحصاری از تصاویر تولید شده توسط generator در همان لحظه، از یک بافر تاریخچه تصاویر استفاده می‌شود که مجموعه‌ای از ۵۰ تصویر تولید شده در تکرارهای گذشته را در خود ذخیره می‌کند. در هر مرحله آموزش، discriminator با ترکیبی از تصاویر جدید و تصاویر موجود در این بافر به‌روزرسانی می‌شود.

این مکانیزم برای حل مشکل Model Oscillation طراحی شده است. اگر discriminator فقط با جدیدترین تصاویر آموزش ببیند، ممکن است به سرعت روی ویژگی‌های خاص نسخه فعلی

generator دچار overfit شود و generator و discriminator وارد چرخه‌ای از تغییرات مداوم شوند بدون اینکه به یک نقطه تعادل همگرا شوند. بافر تاریخچه باعث می‌شود discriminator توزیع کلی‌تری از تصاویر تولید شده در طول زمان را ببیند و فرآیند آموزش پایدارتر شود.

زیربخش سوم : محدودیت های مدل

7

مدل CycleGAN بیشتر در تسک هایی که شامل تغییرات بافت و رنگ هستند، مانند تبدیل نقاشی به عکس یا تغییر فصل، عملکرد بسیار موفقی دارد. با این حال، در تسک هایی که نیازمند تغییرات هندسی ساختاری و بزرگ هستند، مانند تبدیل سگ به گربه یا تبدیل سیب به پرتقال با تغییر شکل قابل توجه، عملکرد ضعیفی از خود نشان می‌دهد. منظور از این محدودیت آن است که مدل تمایل دارد ساختار اصلی و خطوط مرزی اشیاء در تصویر ورودی را تا جایی که می‌تواند حفظ کند و تنها پوسته یا بافت آن را تغییر دهد؛ بنابراین اگر هدف تبدیل، نیازمند تغییر شکل فیزیکی شیء (مثلا تغییر پوزیشن بدن یا ابعاد اعضای صورت) باشد، CycleGAN اغلب شکست می‌خورد.

دلیل اصلی این محدودیت، اعمال سخت گیرانه قید Cycle Consistency Loss است. این قید، مدل را مجبور می‌کند که پس از تبدیل تصویر به دامنه مقصد و بازگرداندن آن، دقیقا به همان تصویر پیکسلی اولیه برسد؛ یعنی داشته باشیم $F(G(x)) \approx x$. اگر generator تغییرات هندسی بزرگی در تصویر ایجاد کند، بطور مثال بدن سگ را کشیده‌تر کند تا شبیه گربه شود، بازگرداندن دقیق آن به حالت اولیه برای generator معکوس بسیار دشوار خواهد بود، زیرا اطلاعات مکانی پیکسل ها جابجا شده است. در نتیجه، برای مینیمم کردن loss چرخه، مدل ترجیح می‌دهد تغییرات هندسی را به حداقل برساند و ساختار مکانی ورودی را بدون تغییر باقی بگذارد، که این امر مانع از یادگیری تغییرات شکلی بزرگ می‌شود.

بخش دوم : پیاده سازی عملی

زیربخش اول : آماده سازی داده ها

در این قسمت، از دیتاست horse2zebra استفاده کردم؛ در ادامه تصاویر نمونه ای از هر دو کلاس horse و zebra آورده شده است.

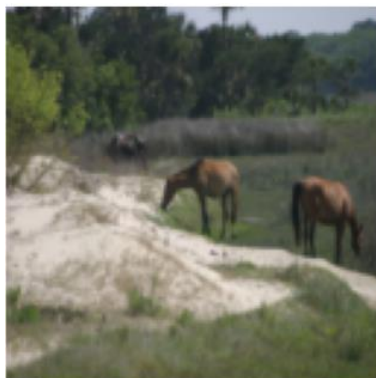
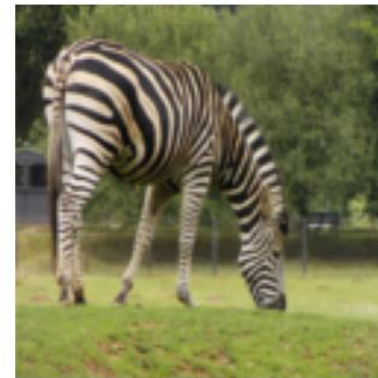
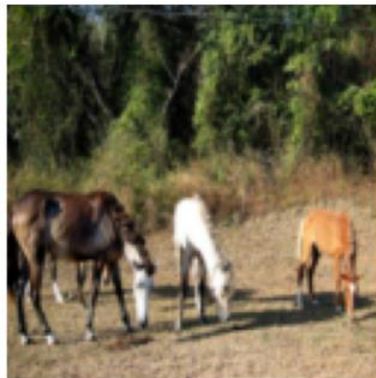


Figure 8 تعدادی تصویر نمونه از دامنه های horse (تصاویر سمت چپ) و zebra (تصاویر سمت راست)

زیربخش دوم : پیاده سازی معماری مدل

پیاده سازی های این قسمت در نوت بوک CycleGAN قابل مشاهده هستند.

زیربخش سوم : پیاده سازی تابع هدف و آموزش مدل

مقادیر ابرپارامترها و توجیه انتخاب:

در فرآیند آموزش مدل CycleGAN ، مقادیر ابرپارامترها را دقیقاً بر اساس پیکربندی پیشنهادی در مقاله اصلی (Zhu et al., 2017) تنظیم کردم تا پایداری آموزش تضمین شود. مقادیر استفاده شده به شرح زیر است:

- تعداد ایپاک‌ها : 20
- **Learning Rate**: مقدار ثابت 0.0002 برای 10 ایپاک اول، و کاهش خطی تا صفر در 10 ایپاک دوم.
- **Batch Size**: 1
- استفاده از Instance Normalization و جلوگیری از میانگین‌گیری دسته‌ای که برای تولید تصویر مناسب‌تر است.
- λ_{cycle} : 10
- اهمیت بالاتر برای حفظ محتوا نسبت به تغییر سبک.
- $\lambda_{identity}$: 5
- معادل نصف λ_{cycle} ، برای حفظ ترکیب‌بندی رنگی.
- **Optimizer**: Adam با پارامترهای $\beta_1 = 0.5$ و $\beta_2 = 0.999$.
- انتخاب این مقادیر بر این مبنا صورت گرفت که این تنظیمات در مقاله اصلی به عنوان مقادیر بهینه برای دیتاست‌های object-to-object (مانند اسب به گورخر) معرفی شده‌اند و تعادل مناسبی میان سرعت همگرایی و کیفیت بصری ایجاد می‌کنند.

نمودارهای loss و تحلیل آنها:

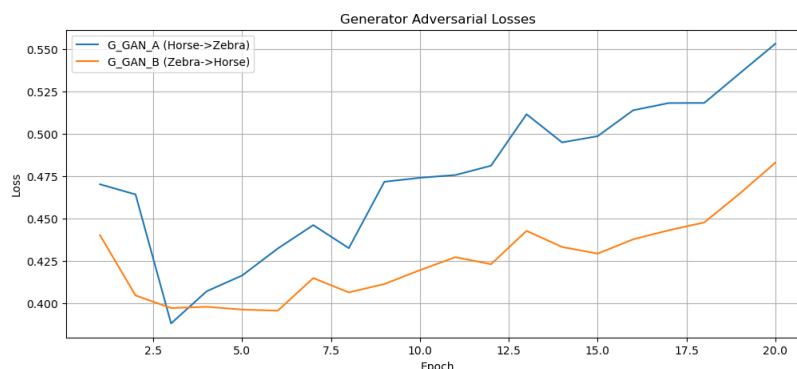


Figure 9 روند تغییرات Adversarial Loss برای generatorها ($G_{A \rightarrow B}$ و $G_{B \rightarrow A}$) در طول 20 ایپاک.

تحلیل Figure 9: این نمودار عملکرد generatorها در فریب دادن discriminatorها را نشان می‌دهد. همانطور که مشاهده می‌شود، مقدار loss برای هر دو مولد روندی نسبتاً صعودی یا نوسانی دارد (از حدود 0.4 به حدود 0.55). این رفتار در شبکه‌های GAN کاملاً مطلوب است؛ زیرا نشان می‌دهد که با گذشت زمان و قوی تر شدن discriminatorها، کار برای generatorها

سخت تر شده است. این رقابت باعث می‌شود generator ها مجبور شوند جزئیات دقیق تری تولید کنند. عدم کاهش این زیان به سمت صفر و وجود نوسانات، نشان‌دهنده پویایی و فعال بودن جریان آموزش تخصصی است. این رفتار بیانگر آن است که discriminator به طور مداوم چالش های جدیدی برای generator ایجاد می‌کند و مانع از همگرایی زودرس مدل به یک جواب بی کیفیت و حتی مشکل mode collapse می‌شود.

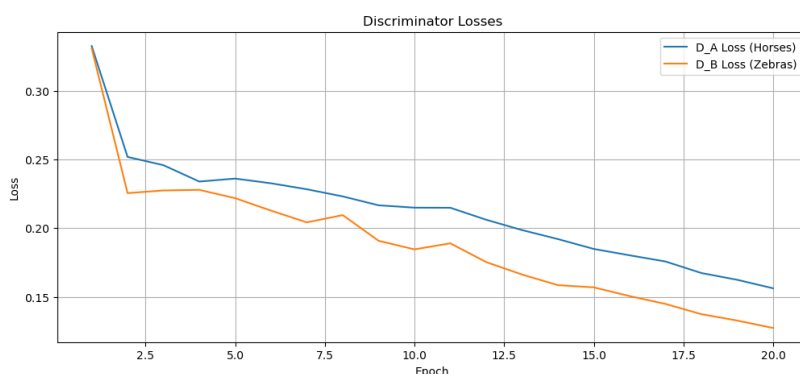


Figure 10 روند کاهش زیان شبکه‌های discriminator (D_A برای تشخیص اسب واقعی و D_B برای تشخیص گورخر واقعی).

تحلیل Figure 10: در این نمودار، توانایی شبکه های discriminator در تفکیک تصاویر واقعی از جعلی نمایش داده شده است. روند نزولی واضح (از ۰.۳۳ به حدود ۰.۱۵) نشان می‌دهد که D_A و D_B در طول زمان یاد گرفته‌اند ویژگی های تصاویر تولید شده را بهتر شناسایی کنند. این کاهش زیان در تقابل با افزایش زیان generator ها (Figure 9) قرار دارد که نشان دهنده شکل گیری یک بازی Minmax سالم است. نکته مهم این است که زیان به صفر مطلق نرسیده، که نشان می‌دهد generator همچنان چالش برانگیز باقی مانده‌اند.

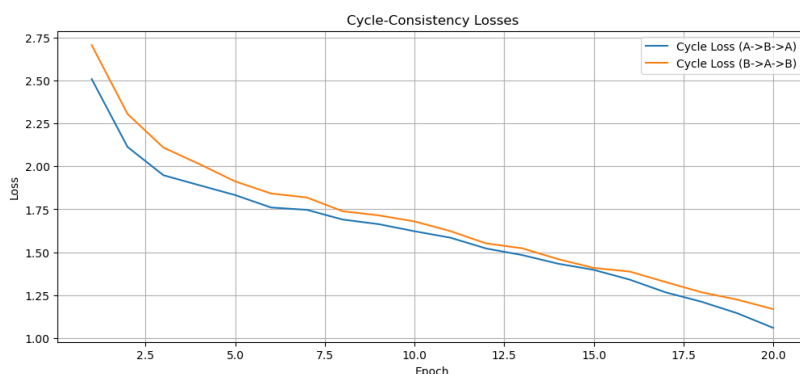


Figure 11 روند همگرایی زیان سازگاری چرخه (Cycle Loss) برای هر دو مسیر بازسازی ($A \rightarrow B \rightarrow A$ / $B \rightarrow A \rightarrow B$)

تحلیل Figure 11: این نمودار حیاتی ترین معیار برای سنجش موفقیت مدل در حفظ محتوای تصویر است. کاهش شدید و پیوسته این زیان (از مقدار اولیه ۲.۵ به حدود ۱.۰) نشان می‌دهد که مدل با موفقیت یاد گرفته است که نگاشت ها را به صورت معکوس پذیر انجام دهد. به عبارت دیگر، مدل یاد گرفته است که هنگام تبدیل اسب به گورخر، ساختار بدن و پوزیشن حیوان را حفظ کند تا بتواند

آن را در مسیر برگشت دقیقاً بازسازی نماید. همگرایی این نمودار تضمین کننده کیفیت ساختاری تصاویر خروجی است.

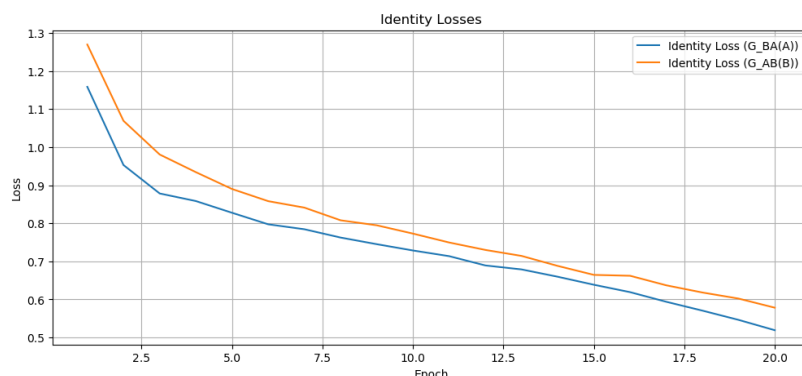


Figure 12 روند کاهش زیان همانی (Identity Loss) در طول فرآیند آموزش.

تحلیل **Figure 12**: این نمودار که مربوط به بخش تکمیلی پروژه است، نشان می‌دهد مدل تا چه حد در حفظ ویژگی‌های ذاتی تصاویر (زمانی که نیازی به تغییر نیست) موفق بوده است. روند نزولی (از ۱.۲ به ۰.۵) بیانگر این است که generator ها یاد گرفته اند اگر تصویر مقصد (مثلاً گورخر) به عنوان ورودی به آن‌ها داده شود، نباید تغییری در آن ایجاد کنند. این ویژگی باعث شده است که در تصاویر نهایی، ترکیب‌بندی رنگی و نور پردازی صحنه حفظ شود و از تغییرات رنگی غیر واقعی جلوگیری گردد.

خروجی‌های نمونه مدل:

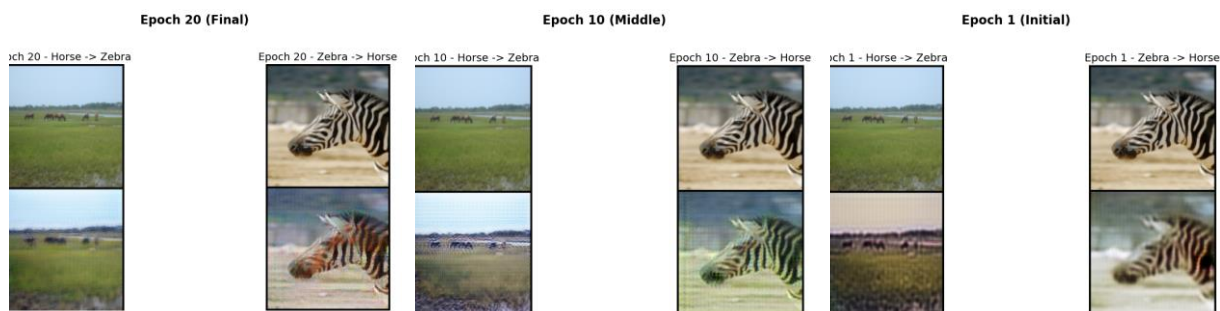


Figure 13 روند تکامل خروجی‌های مدل در فاز آموزش (اپیک‌ها ۱۰، ۱ و ۲۰).



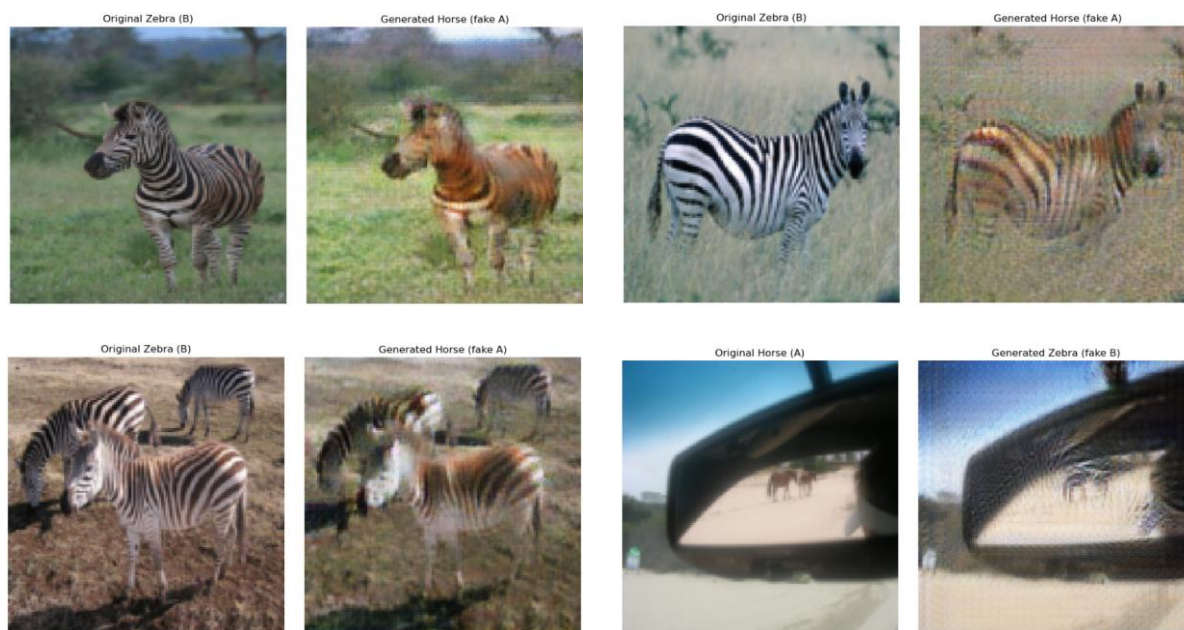
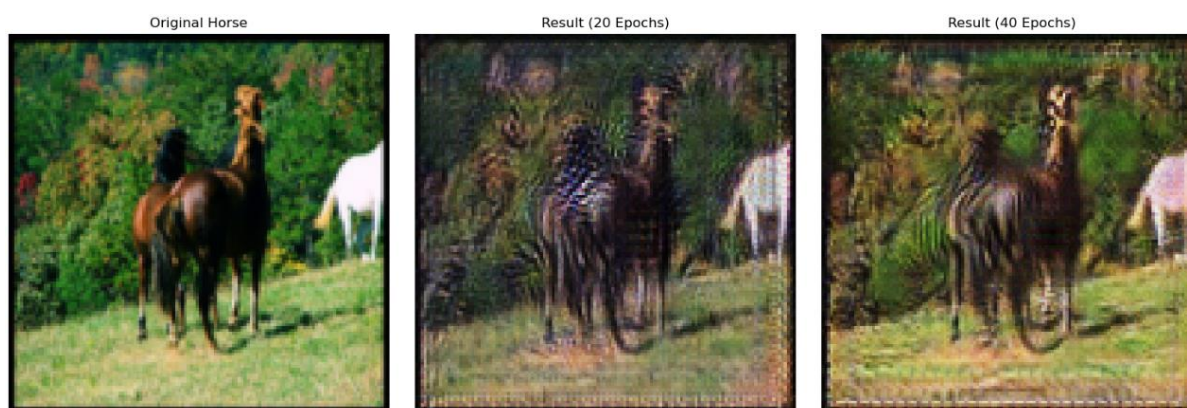


Figure 14 نمونه هایی از خروجی مدل پس از 20 اپیاک آموزش

بخش تکمیلی: بررسی تأثیر افزایش زمان آموزش تا ۴۰ اپیاک

اگرچه در صورت پروژه آموزش مدل به مدت ۲۰ اپیاک خواسته شده بود، بررسی نتایج اولیه در پایان این بازه نشان داد که مدل هنوز به همگرایی کامل نرسیده است. با وجود کاهش چرخه Cycle Loss، تصاویر خروجی در اپیاک ۲۰ همچنان دارای نویز و آرتیفکتهایی بودند که نشان‌دهنده پتانسیل مدل برای یادگیری بیشتر بود. به همین منظور، تصمیم گرفته شد فرآیند آموزش از آخرین نقطه ذخیره شده ادامه یابد و مدل برای ۲۰ اپیاک اضافی (در مجموع ۴۰ اپیاک) تحت آموزش قرار گیرد. هدف از این آزمایش، بررسی این فرضیه بود که آیا افزایش زمان آموزش در مدل‌های CycleGAN می‌تواند بدون تغییر در معماری یا ابرپارامترها، منجر به بهبود محسوس در کیفیت بصری و ثبات بافت ها شود یا خیر. در ادامه، خروجی‌های مدل در اپیاک ۴۰ و مقایسه آن با خروجی های مدل در مراحل را نشان داده ام.



Original Zebra



Result (20 Epochs)



Result (40 Epochs)



Original Horse



Result (20 Epochs)



Result (40 Epochs)



Original Zebra



Result (20 Epochs)



Result (40 Epochs)



Original Horse



Result (20 Epochs)



Result (40 Epochs)



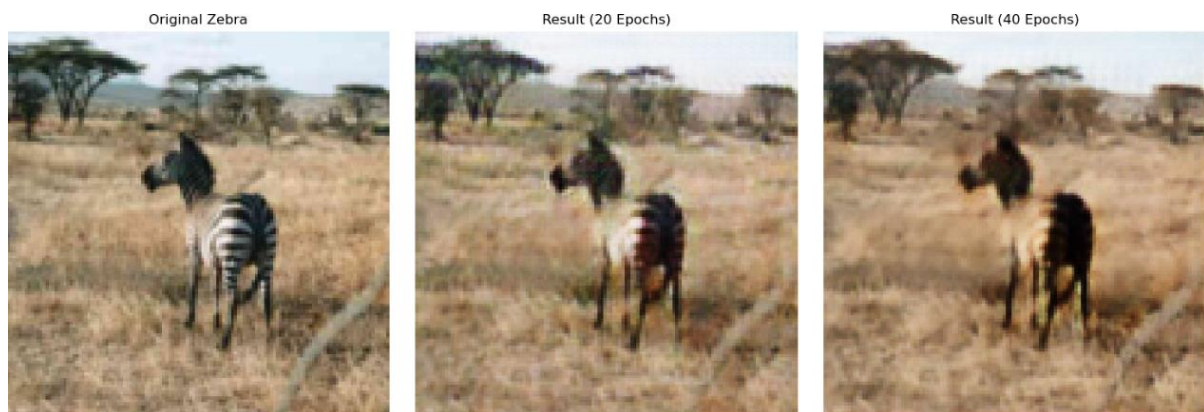


Figure 15 نمونه هایی از خروجی های مدل در شروع، اپیاک 20 و اپیاک 40

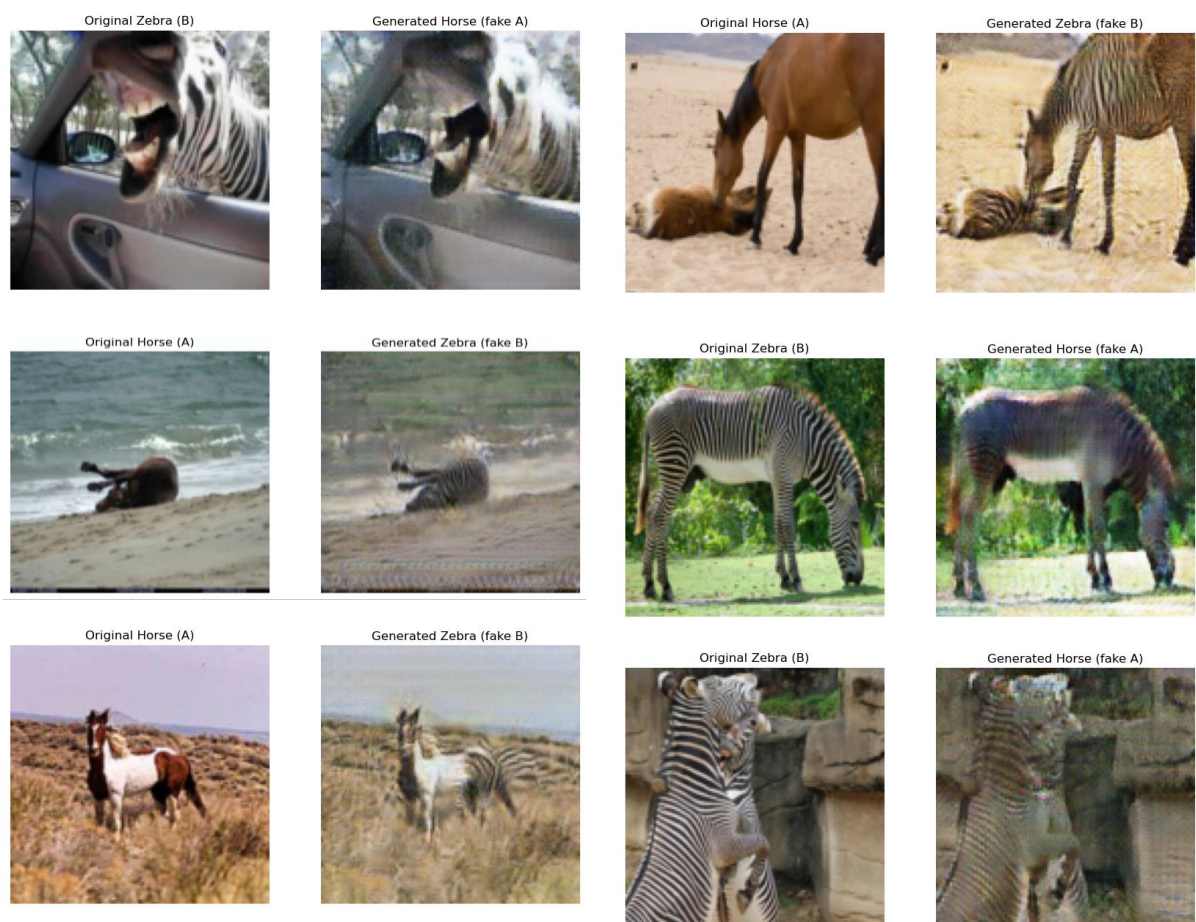


Figure 16 نمونه خروجی های مدل پس از 40 اپیاک آموزش

- Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). Image-to-image translation with conditional adversarial networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017) (pp. 1125–1134).
- Zhu, J. Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. In Proceedings of the IEEE International Conference on Computer Vision (ICCV 2017) (pp. 2223–2232).
- Papamakarios, G., Pavlakou, T., & Murray, I. (2017). Masked Autoregressive Flow for Density Estimation. In *Advances in Neural Information Processing Systems (NIPS)*.
- Germain, M., Gregor, K., Murray, I., & Larochelle, H. (2015). MADE: Masked Autoencoder for Distribution Estimation. In *International Conference on Machine Learning (ICML)*.
- Kingma, D. P., Salimans, T., Jozefowicz, R., Chen, X., Sutskever, I., & Welling, M. (2016). Improving Variational Inference with Inverse Autoregressive Flow. In *Advances in Neural Information Processing Systems (NIPS)*.
- <https://tech.skit.ai/normalizing-flows-part-2/>